

LOCAL MACHINE SETUP

Resource Requirements Assessment

Title	Resource Requirements Assessment
Department	Data Engineering
Date	June 15, 2026
Prepared By	Bezawit Assefa

1. Purpose

This document outlines the software and hardware requirements for a local development machine in a Data Engineering role. It covers the tools needed to build, test, and iterate on data pipelines, SQL transformation layers, and basic machine learning experiments locally. The goal is to help the team evaluate whether a given machine is adequately provisioned for this work and to identify gaps that may limit productivity.

Specifications are organized into Minimum, Recommended, and Advanced/Optional tiers where relevant. Minimum represents a workable baseline; Recommended reflects what enables comfortable day-to-day work without significant constraint; Advanced/Optional covers capabilities that are useful but not required in a standard internship or junior data engineering context.

2. Data Engineering & Processing Stack

The tools below form the core of a local data engineering environment. Each is chosen based on industry adoption, production relevance, and compatibility with the broader team stack.

2.1 Python — Environment Management

Python is the primary language for data engineering, pipeline scripting, and ML experimentation. Environment management is critical to prevent dependency conflicts across projects.

- `pyenv`: Manages multiple Python versions; allows per-project version pinning.
- `Poetry` or `pip-tools`: Deterministic dependency resolution with lockfile support for reproducible environments.
- Recommended Python version: 3.10 or 3.11. These versions provide the best balance of stability, package compatibility, production readiness, and ecosystem support for data engineering and machine learning workloads.

2.2 PostgreSQL — Relational Database

PostgreSQL is the recommended local database for pipeline validation, dbt target configuration, and general SQL workloads. It should be run via Docker to keep the host environment clean.

- Supports JSONB, window functions, CTEs, and full-text search.
- Compatible with dbt-postgres adapter out of the box.
- Version 15 or 16 recommended.

2.3 Docker — Containerization

Docker enables consistent, isolated environments for all services used in local development. It eliminates setup inconsistencies across machines and mirrors the containerized nature of most production deployments.

- `Docker Compose`: Manages multi-service setups (e.g., Airflow + PostgreSQL + Redis) with a single configuration file.

- Container images built and validated locally transfer cleanly to staging or production environments.

2.4 Airflow / Dagster — Pipeline Orchestration

A workflow orchestrator is needed to schedule, monitor, and manage dependencies between pipeline steps. Both options are appropriate depending on team preference.

- Apache Airflow (2.8+): Mature, widely used, large provider ecosystem. Good choice if the team already uses Airflow.
- Dagster (1.6+): Asset-centric model with strong typing and a better local development experience. Good choice for greenfield projects.
- Both can run locally via Docker Compose and do not require cloud resources for development.

2.5 dbt — SQL Transformation Layer

dbt introduces software engineering practices into SQL transformations: version control, modular models, testing, and documentation. It runs locally against a configured database target.

- dbt Core (open source) with the dbt-postgres adapter is sufficient for local development.
- Version 1.7 or later recommended.
- Integrates directly with Airflow/Dagster for end-to-end ELT workflows.

Tool / Library	Version	Purpose
Python + pyenv	3.10 / 3.11	Language runtime and environment management
PostgreSQL	15 / 16 (via Docker)	Local relational database; dbt target
Docker + Compose	Docker 25+	Service containerization; environment consistency
Apache Airflow	2.8+	DAG-based workflow scheduling and monitoring
Dagster	1.6+	Asset-centric pipeline orchestration (alternative to Airflow)
dbt Core	1.7+	SQL transformation, testing, and documentation

3. Small Language Model (SLM) Experimentation Setup

This section applies if the role involves local ML experimentation or fine-tuning small language models. These requirements are considered Advanced/Optional for a standard data engineering role and are not needed for pipeline or SQL work.

3.1 GPU and VRAM Requirements

GPU VRAM is the primary constraint for local SLM work. Requirements vary by task:

- Inference only (no training): 6–8 GB VRAM with 4-bit quantization (e.g., GTX 1660 or RTX 3060). Sufficient for testing model outputs locally.
- Fine-tuning via QLoRA (7B model): 8–10 GB VRAM recommended. Enables parameter-efficient fine-tuning without loading the full model in fp32.
- Full LoRA fine-tuning or larger models: 16–24 GB VRAM. This is not a requirement for an internship context; cloud resources (Colab Pro, AWS Spot) are more cost-effective at this scale.
- CUDA Toolkit 12.1+ and a compatible NVIDIA driver are required for PyTorch GPU support.

3.2 Core ML Libraries

The following libraries support local SLM experimentation with reasonable hardware requirements:

Tool / Library	Version	Purpose
PyTorch	2.2+ (CUDA 12.1)	Core training framework; GPU-accelerated tensor operations
transformers	4.38+	Model loading, tokenization, and inference for HF models
peft	0.9+	LoRA / QLoRA adapters for parameter-efficient fine-tuning
bitsandbytes	0.43+	4-bit / 8-bit quantization to reduce VRAM usage
datasets	2.17+	Streaming dataset loading and preprocessing
accelerate	0.27+	Mixed-precision and device abstraction for training
trl	0.8+	SFT (Supervised Fine-Tuning) training loops; RLHF optional
wandb / MLflow	Latest	Experiment tracking; optional but strongly recommended

Note: Advanced techniques such as RLHF and DPO are not recommended as starting points for local experimentation. SFT with LoRA/QLoRA is the appropriate scope for an internship-level setup.

4. Data Visualization & Exploration Tools

4.1 Exploratory Analysis

- JupyterLab: Standard environment for data exploration, EDA, and iterative analysis. Integrates directly with pandas, Matplotlib, and Plotly.
- pandas: Primary DataFrame library for structured data manipulation.
- polars: Rust-based DataFrame library; useful for large files that are slow to process in pandas.
- DuckDB: In-process SQL engine. Allows SQL queries directly on Parquet and CSV files without a database server. Useful for ad-hoc analysis in notebooks.

4.2 Visualization Libraries

- Plotly: Interactive charts that render in Jupyter and export to HTML. Good for sharing standalone analysis outputs.
- Seaborn + Matplotlib: Statistical visualization for distribution plots, heatmaps, and correlation analysis.
- Altair: Grammar-of-graphics approach; concise and well-suited to time series and multi-variable charts.

4.3 Dashboards and Reporting

- Streamlit: Converts Python scripts into simple web apps. Useful for building internal data tools or model demo interfaces without frontend work.
- Grafana (via Docker): Time series monitoring and pipeline health dashboards. Connects natively to PostgreSQL.
- Power BI Desktop (optional): For stakeholder-facing reports requiring enterprise formatting or DirectQuery integration with existing BI infrastructure.

5. Hardware Requirements

The table below describes the hardware needed to run the data engineering stack described in this report. GPU requirements apply only if SLM experimentation is in scope.

Component	Minimum	Recommended	Notes
RAM	16 GB DDR4	32 GB DDR4/5	16 GB is viable for pipeline dev and light containerization. 32 GB is preferred for running Airflow + PostgreSQL + dbt simultaneously alongside SLM inference.
CPU	6-core (e.g. i5 / Ryzen 5)	12-core (e.g. i7 / Ryzen 7)	Higher core count benefits multi-threaded dbt compilation, parallel Airflow task execution, and Docker build times.
GPU	None / GTX 1660 (6 GB VRAM)	RTX 3070/3080 (8–10 GB VRAM)	Required only for SLM fine-tuning. GTX 1660 supports 4-bit inference; 8–10 GB VRAM enables QLoRA fine-tuning of 7B models.
Storage	256 GB NVMe SSD	512 GB+ NVMe SSD	Docker images, Python environments, and local datasets accumulate quickly. NVMe ensures fast container startup and data loading.
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS (native)	Native Linux avoids Docker Desktop overhead and ensures CUDA driver compatibility with production tooling.

* GPU is not required for data engineering pipeline work. It becomes relevant only when local SLM fine-tuning is part of the role. Cloud-based GPU resources are a practical alternative for occasional training runs.

6. Resource Justification

The software stack described in this report places real demands on local hardware. The following explains the practical basis for each hardware requirement.

RAM (32 GB Recommended)

- Running Airflow or Dagster in Docker alongside PostgreSQL, a dbt compilation process, and a JupyterLab session concurrently can consume 10–14 GB of RAM under moderate load.
- 16 GB is a workable minimum but leaves limited headroom. 32 GB ensures stable multi-service operation without memory pressure affecting task execution.

NVMe SSD

- Docker images for data engineering tools range from 500 MB to several GB each. Python virtual environments, local datasets, and model checkpoints add further storage overhead.
- An HDD introduces significant latency during container startup, large file reads, and dataset loading. An NVMe SSD is the most impactful single hardware upgrade for day-to-day workflow speed.

CPU (Multi-Core)

- dbt compiles and runs SQL models in parallel. Airflow schedules concurrent task runs. Docker build processes use multiple threads.
- A 4-core machine can handle these tasks but will show latency under load. A 12-core machine handles concurrent workloads without becoming a bottleneck.

GPU (Conditional)

- No GPU is needed for pipeline development, SQL work, or data visualization.
- If local SLM experimentation is required, a consumer GPU with 8–10 GB VRAM (e.g., RTX 3070 or 3080) enables QLoRA fine-tuning of 7B models.
- Higher-end hardware such as the RTX 4090 is not justified for an internship-level setup; cloud resources should be used for larger training runs.

OS (Ubuntu 22.04)

- Docker, CUDA, and most data engineering tooling are developed and tested on Linux. Native Ubuntu avoids the performance and compatibility overhead of Docker Desktop on Windows or macOS.

- WSL2 or macOS are acceptable short-term workarounds but may require additional configuration for CUDA and some orchestration tools.

7. Implementation Approach

The following phased sequence is recommended for setting up the environment progressively, validating each layer before adding the next.

Phase 1 — Foundation

- Install Ubuntu 22.04, configure pyenv, set up Poetry or pip-tools, and deploy PostgreSQL via Docker.
- Validate by running a basic dbt project against the local database.

Phase 2 — Orchestration

- Deploy Airflow or Dagster using Docker Compose.
- Build and schedule a sample multi-step pipeline.
- Confirm dbt integration within the orchestration layer.

Phase 3 — Exploration Tools

- Set up JupyterLab, install pandas/polars/DuckDB, and configure Plotly or Seaborn for visualization.
- Build a simple Streamlit dashboard connected to the PostgreSQL instance.

Phase 4 — ML Experimentation

- Configure CUDA, install PyTorch and the Hugging Face stack, and run a baseline inference test.
- Proceed to SFT fine-tuning using QLoRA on a small dataset with W&B tracking enabled.